# Seed 404 — `v04_tuned`

What changed, what win rate to expect, and why

Team Seed 404 (Sanyam Agarwal), Hyde × WnCC Catan AI Hackathon, IIT Bombay

17 October 2026, 15:05 IST

## Summary

`v04_tuned` keeps `v01_catan_fixes`' one-move lookahead and replaces its hand-set weights with weights **found by an evolution strategy against our whole opponent pool**. On 1,500 fresh local games it beats `v01` against every opponent and wins **61.0% ± 5.5** head to head against `v01`. The requested target was 70% against F. **We do not expect 70% against F**: our estimate is about 57–58% on the organisers' judge. Section 3 explains why, and Section 4 what would be needed to get closer.

## 1 What is different from v01

1. **Every weight is tunable.** F's 12 value-function weights (VP weight fixed at $3 \times 10^{14}$) and v01's 4 Catan fixes (2:1 port, 3:1 port, road progress, army progress) are read from `weights.json` beside `agent.py`. If the file is missing or broken the agent falls back to v01's weights.
2. **Two new relative features** that let the tuner react to the opponent: `road_race` = (our longest road − theirs), clipped to ±5, once theirs is ≥ 4; and `opp_vps` = a penalty per opponent public victory point, so taking Longest Road or Largest Army *from* them counts twice.
3. **The weights were tuned** with `tools/tune.py` (new, in the repo):
   - (1+$\lambda$) evolution strategy, $\lambda = 10$ mutants per generation; each mutant multiplies 1–3 weights by $e^{\mathcal{N}(0,\,0.7)}$ (log-scale steps, signs kept).
   - Fitness = mean win rate against F, `road_army_f`, `card_hoarder` and `v01`; 40 games per opponent per candidate, seats alternated.
   - All candidates of a generation play the *same* fresh seeds (paired comparison). The best mutant must beat the incumbent there *and* on a second fresh seed block before it is accepted (protects against picking a lucky mutant).
   - Seeds 200000–200840 only (never the 100000 quick-check or 990000 judge seeds). 45 minutes on 11 cores: 11 generations, about 13,000 games, 9 accepted steps. Every candidate is logged in `tools/tune_log.csv`.
4. Nothing else changes: same search (one move ahead, one sampled world), same CPU as v01 (about 200 ms per game; 0 errors, 0 games over budget in local runs).

**What the tuner changed**

| Weight | v01 | v04 | Reading |
|---|---|---|---|
| `enemy_production` | $-1.0 \times 10^8$ | $-2.6 \times 10^9$ | $\times 26$: deny the opponent's income |
| `discard_penalty` | $-5$ | $-27$ | avoid holding $> 7$ cards |
| `reachable_production_0` | $1.0 \times 10^4$ | $2.4 \times 10^4$ | value spots we can build now |
| `army_size` | 10.1 | 17.4 | slightly more for knights |
| `num_tiles` | 1 | 2.2 | tie-breaker |
| `hand_resources` | 1 | 0.49 | tie-breaker |
| `port2` | $3.0 \times 10^7$ | $1.4 \times 10^7$ | ports over-valued |
| `port3` | $1.0 \times 10^7$ | $4.6 \times 10^6$ | ports over-valued |
| `opp_vps` | (new) $3 \times 10^{13}$ | $1.1 \times 10^{13}$ | per opponent VP |

The one large change is `enemy_production`. With it, the agent puts the robber on the opponent's best hexes and takes building spots the opponent needs much more aggressively. In a two-player game with no trading between players, every resource the opponent does not collect is a resource they cannot replace; F's weight on it was 26 times too small for this setting.

## 2 Measured so far (local, not yet the organisers' kit)

Sanyam's laptop, catanatron 3.3.0 from GitHub, 300 games per opponent on seeds 400000+ (never seen by the tuner), same seeds for both agents. ci95 $\approx \pm 5.5$ points.

| Opponent | `v04_tuned` | `v01` | Difference |
|---|---|---|---|
| F | 60.3% | 58.0% | +2.3 |
| `road_army_f` (our worst matchup) | 53.0% | 49.0% | +4.0 |
| `card_hoarder` | 73.0% | 68.3% | +4.7 |
| `greedy_builder` | 86.3% | 83.0% | +3.3 |
| `v01` (head to head) | **61.0%** | — | +11 vs. 50% |
| Mean of F, road, hoarder, greedy | 68.2% | 64.6% | +3.6 |
| Worse half (2 lowest of 4) | 56.7% | 53.5% | +3.2 |

Every single difference is in v04's favour. Each one alone is inside the noise, but the head to head against v01 (61.0% $\pm$ 5.5) is clearly above 50%.

## 3 Expected win rate, and why

| Test (organisers' kit) | v01 measured | v04 expected |
|---|---|---|
| Judge vs F (1,000 games, seed 990000) | 55.3% | **57–58%** (range 54–61) |
| Head to head vs v01 (400 games) | — | **57–61%** |
| `road_army_f` / `card_hoarder` / `greedy_builder` | 48.8 / 71.3 / 79.5% | +2 to +5 each |
| Official, vs O (Hyde's bot) | 39.3% | 41–47% (guess) |

- **Why only about +2–3 against F.** Locally, v01 scores 58.0% against F but 55.3% on the organisers' judge: our local harness reads about 3 points high (it hands the agent the real game instead of a sample with re-drawn hidden cards, and the engine is not the organisers' patched build). We assume the same offset for v04, so its 60.3% maps to about 57–58%.

- **Why we expect more against stronger opponents (v01, O).** The gain is largest where the opponent is good at building: against v01, a stronger F-like player, v04 wins 61%. Denying income hurts an opponent that would otherwise out-produce us. O beats F, so we expect part of this to carry over, but we have no way to measure O before uploading.
- **Why not 70% against F.** (1) The agent still looks only one move ahead, like F. The two agents differ only in *how they score* positions, and Catan has a lot of dice luck, so even a clearly better scorer moves the win rate by points, not tens of points: every change measured so far (v01, v02, v03, v04) moved it by 0 to 6 points against F. (2) The tuning was short: 11 generations, each candidate judged on 160 games (±4 points of noise), so only steps of about 3+ points could be detected. (3) More search on the same value function did not help (v03: 55.6%, no change), so search alone will not close the gap.

**Strength score.** Strength $= 0.5 \cdot \text{win}_F + 0.5 \cdot \text{win}_O$. v01 scored $0.5(0.541) + 0.5(0.393) = 0.467$. With the estimates above, v04 would score about $0.5(0.57) + 0.5(0.44) \approx 0.50$.

## 4 What would be needed to approach 70%

1. **More tuning** (cheapest): continue from `weights.json` on the team machine, with `tools/tune.py`, `--start agents/v04_tuned/weights.json`, `--seed0 300000`, `--minutes 40`. More generations and more games per candidate find the smaller steps the first run could not see.
2. **Combine with exact chance expectation** (`v05_expect_relative`) once that is measured, then re-tune the combination.
3. **A learned value function** (Jainam's `vj01` experiment): the only idea with 10+ points of upside, but also the riskiest.

## How to verify (Jainam)

```
python -m catan_kit.evaluate --a ~/Seed-404/agents/v04_tuned/agent.py \
    --b examples/f_agent.py --seed0 100000 --games 400
python ~/Seed-404/opponents/run_pool.py \
    --a ~/Seed-404/agents/v04_tuned/agent.py --games 400
python -m catan_kit.evaluate --a ~/Seed-404/agents/v04_tuned/agent.py \
    --b ~/Seed-404/agents/v01_catan_fixes/agent.py --seed0 100000 --games 400
```

Upload rule: upload v04 if the judge against F is at least v01's 55.3% and the head to head against v01 is above 50% outside the ci95.